

Patterns for highly autonomous agents

Inventory Database

id	name	description	price	stock
1001	Aviator	Timeless pilot style for any occasion, metal frame	80	12
1002	Catseye	Glamorous 1950s profile, plastic frame	60	28
1003	Moon	Oversized round style, plastic frame	120	15
1004	Classic	Classic round profile, gold frame	60	9

Customer query:

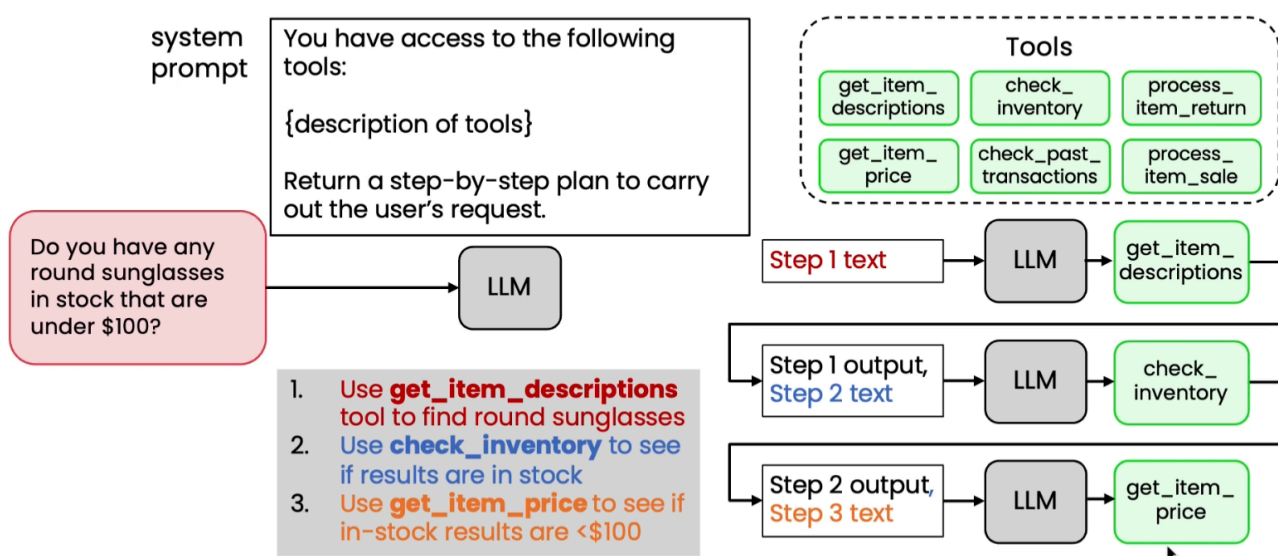
Do you have any round sunglasses in stock that are under \$100?

Yes, we have our **Classic** sunglasses, which are a classic round metal frame and cost \$60

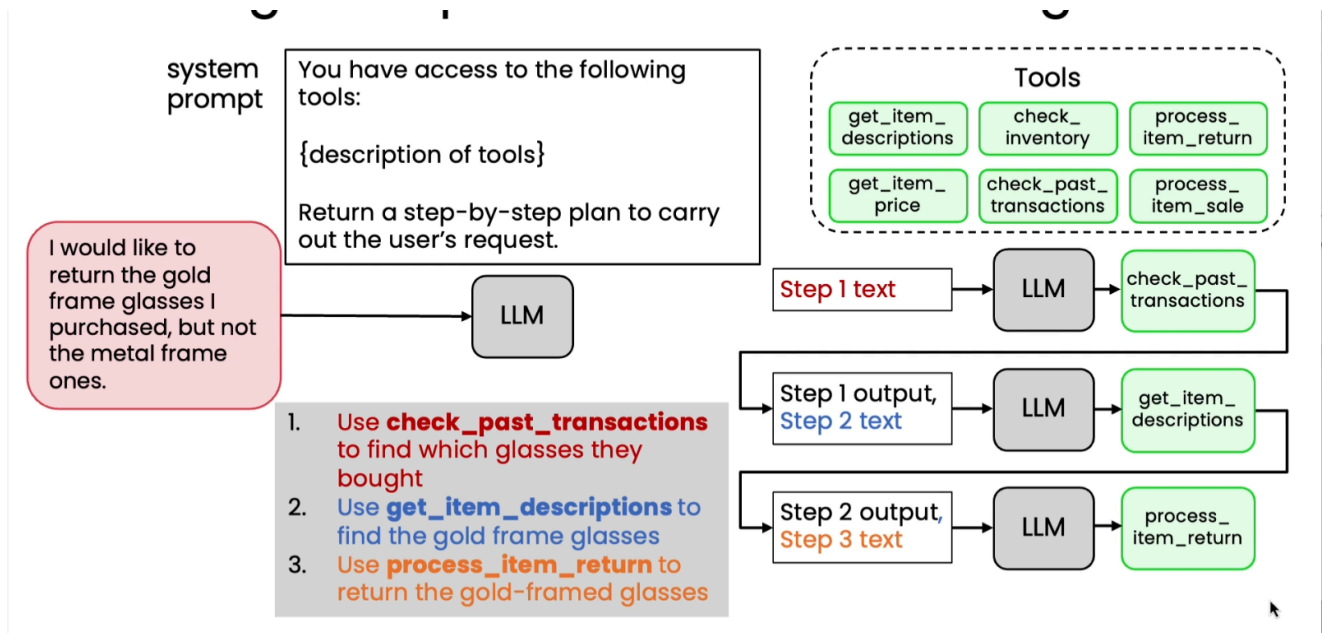
Example: we need to look for round glasses, then see whether it is in stock or not, and finally, we check the price according to the customer's inquiry

Tools:

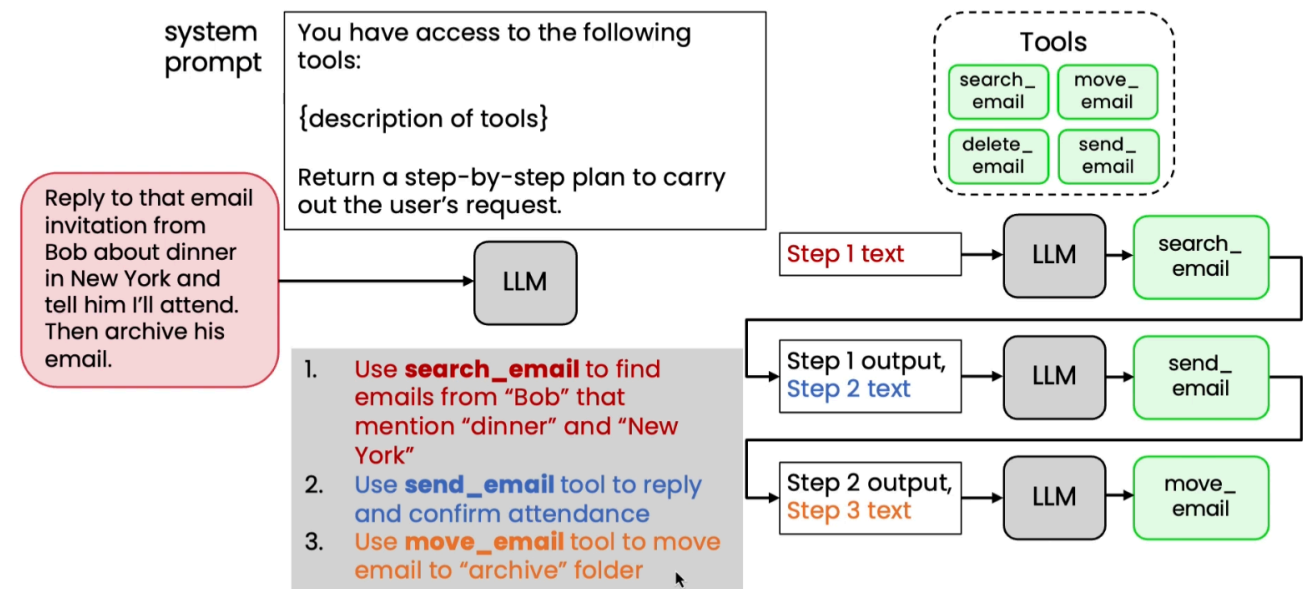
- get_item_description
- check_inventory
- process_item_return
- get_item_price
- check_past_transaction
- process_item_sale



You don't expect the model to go step by step in order, but



more example



Limitation

It makes the system hard to control. we don't really know what plan it will come up with, so we really need to think outside the box

for the agent that plan for themselves, we dont need to hardcoded

Formatting plan

Updated system prompt

You have access to the following tools:

{description of tools}

Create a step-by-step plan in JSON format.

Each step should have the following items:
step number, description, tool name, and args.

Do you have any round sunglasses in stock that are under \$100?

LLM

```
{
  "plan": [
    {
      "step": 1,
      "description": "Find round sunglasses",
      "tool": "get_item_descriptions",
      "args": {"query": "round sunglasses"}
    },
    {
      "step": 2,
      "description": "Check available stock",
      "tool": "check_inventory",
      "args": {"items": "results from step 1"}
    },
    ...
  ]
}
```

Limitation

the developer has to update the tool to be able to catch up with what customer will come up with.

--> but this is tedious, inefficient.

Instead we can ask the model to write code to solve the user's query. Return your answer as python code delimited with `<execute_python>` and `</execute_python>` tags

system prompt

Write code to solve the user's query. Return your answer as python code delimited with `<execute_python>` and `</execute_python>` tags.

What were the amounts of the last 5 transactions?

LLM

```
<execute_python>

import pandas as pd

# Load CSV
df = pd.read_csv("transactions.csv")

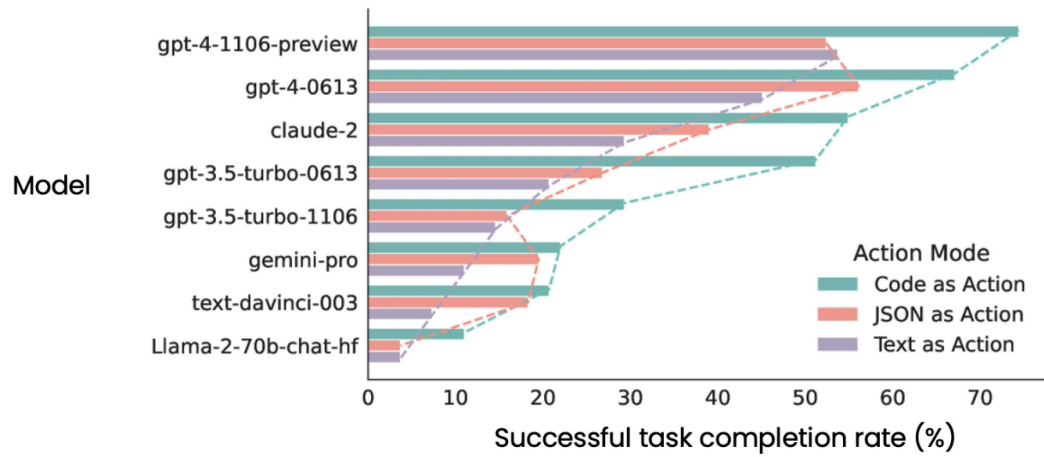
# Ensure date column is parsed as datetime
df["date"] = pd.to_datetime(df["date"])

# Sort by date to get most recent transactions
df_sorted = df.sort_values(by="date",
                           ascending=False)

# Select the last 5 transactions
last_5 = df_sorted.head(5)

# Show just the price column (amounts)
print("Last 5 transaction amounts:")
print(last_5["price"].to_list())

</execute_python>
```



[Adapted from "Executable Code actions Elicit Better LLM Agents", Wang et al. 2024]

The model (filter-->compute--->update)

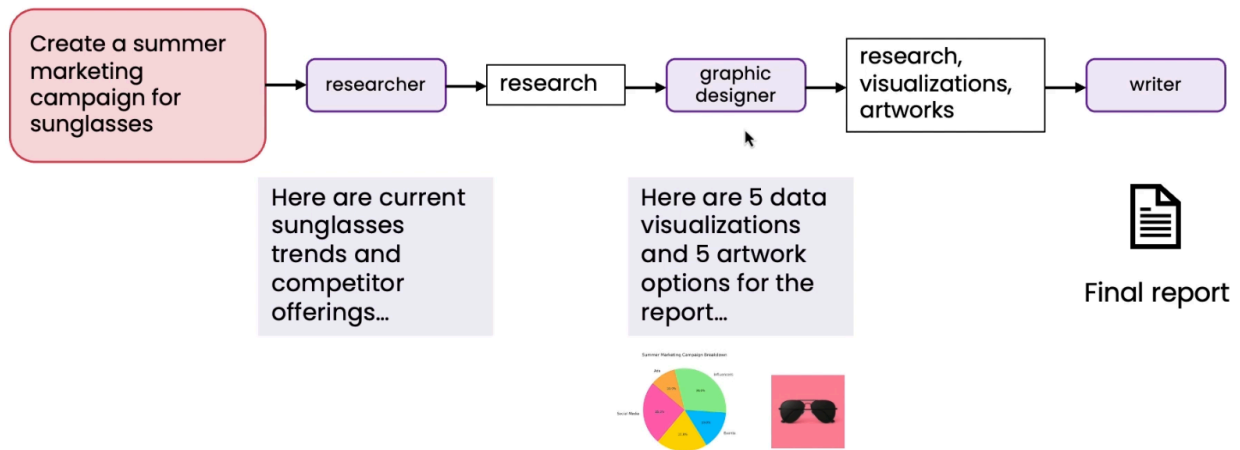
Multi-agentic workflow

that help give another way to tackle complex tasks

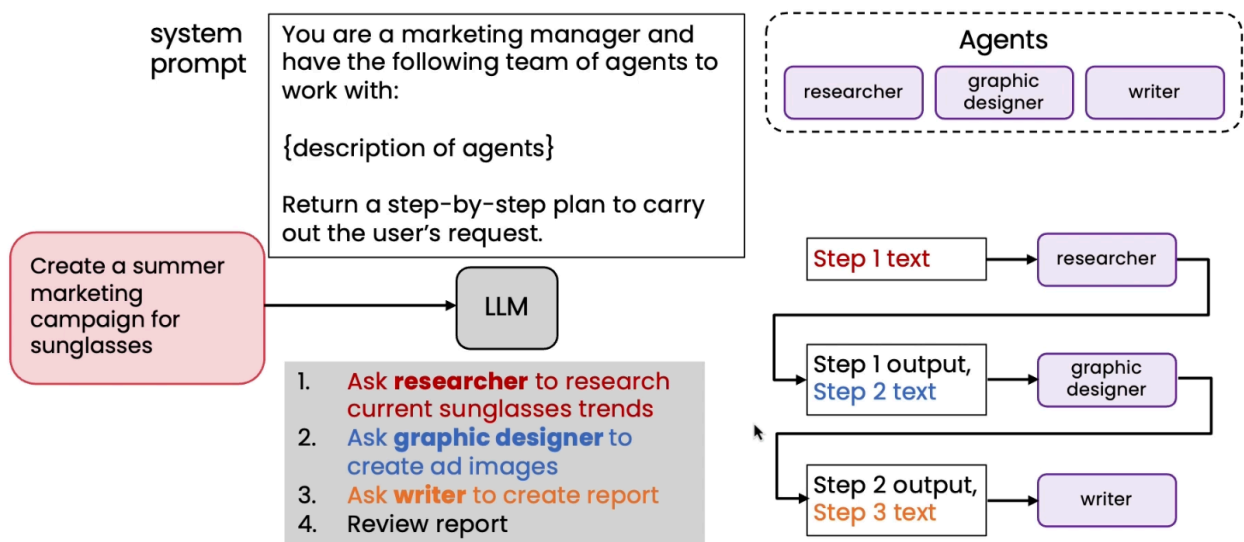
Task	Team
Create marketing assets	Researcher Graphic Designer Writer
Writing a research article	Researcher Statistician Lead writer Editor
Preparing a legal case	Associate Paralegal Investigator

We prompt the agent to play the role of a researcher, a writer, or a graphic designer.

Example: Marketing team with linear plan

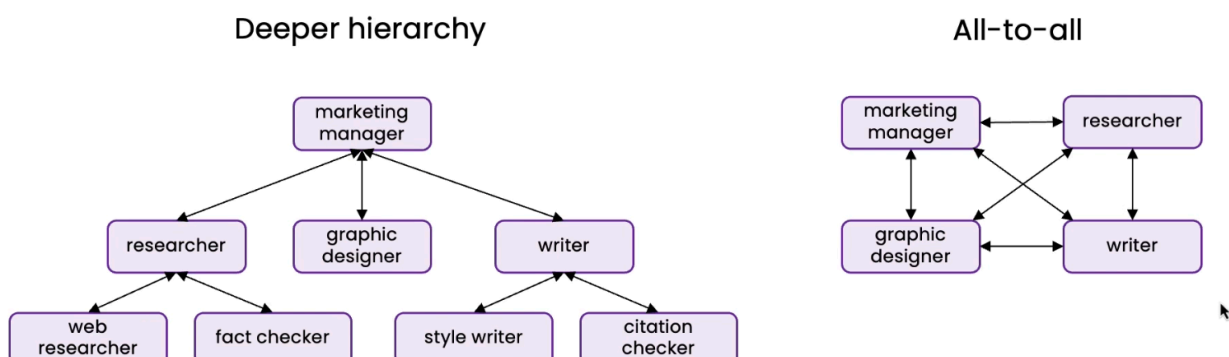


Example: Planning with multiple agents



The marketing manager is the one who monitors or coordinate the steps

Designing the Communication patterns for Multi-agent



All-to-all: Every agent can talk to every agent; the result is hard to predict.